法律声明

- 本课件包括演示文稿、示例、代码、题库、视频和声音等内容，小象学院和主讲老师拥有完全知识产权的权利；只限于善意学习者在本课程使用，不得在课程范围外向任何第三方散播。任何其他人或机构不得盗版、复制、仿造其中的创意及内容，我们保留一切通过法律手段追究违反者的权利。

- 课程详情请咨询
  - 微信公众号：小象
  - 新浪微博：ChinaHadoop

梯度下降和拟牛顿

小象学院 ChinaHadoop.cn

邹博

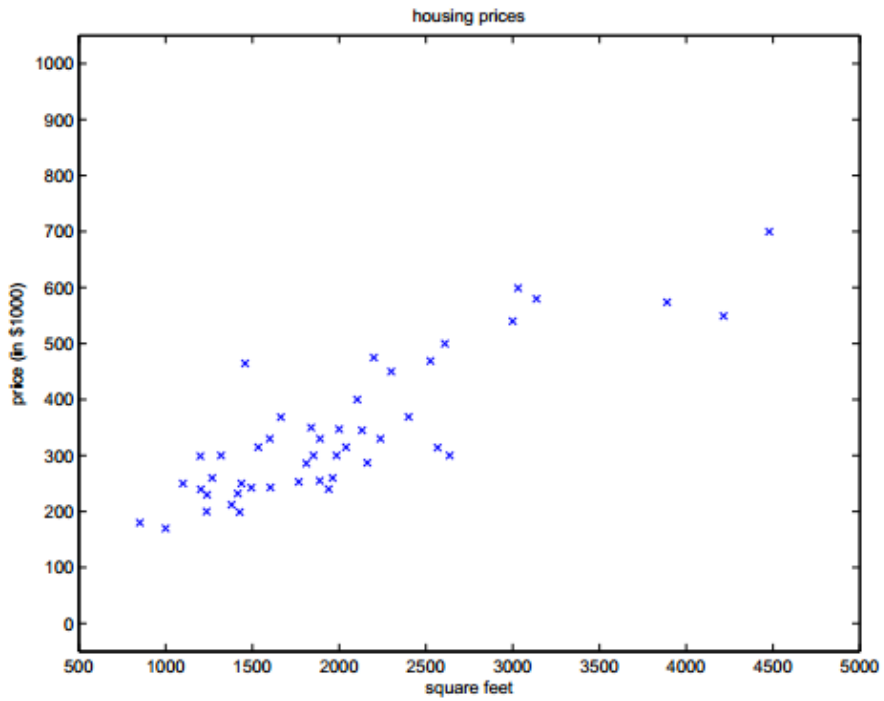
预备题目

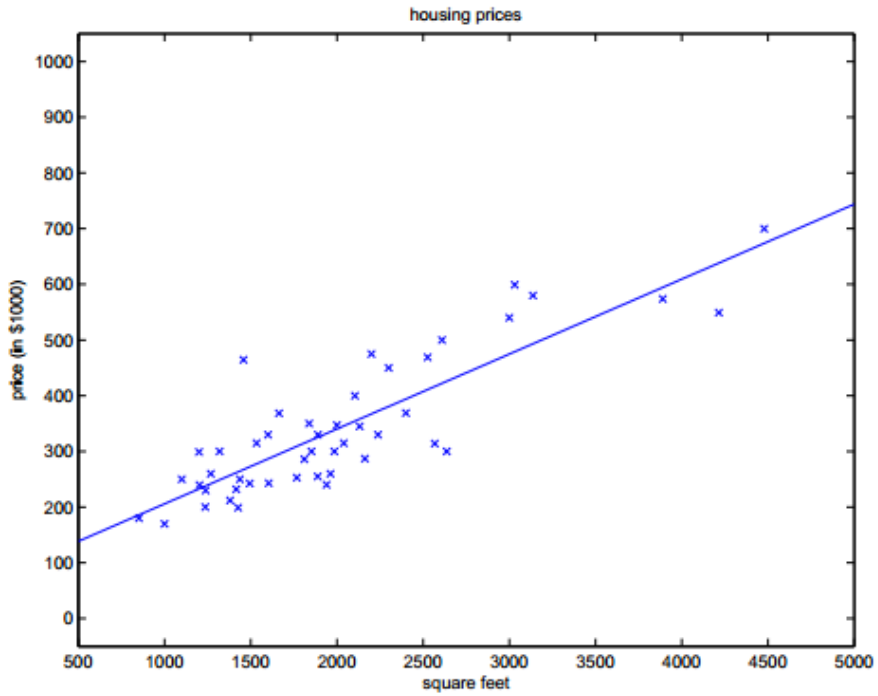
- 已知二次函数的一个点函数值和导数值，以及另外一个点的函数值，如果确定该函数的解析式？
  - 即：二次函数f(x)，已知f(a)，f’(a)，f(b)，求f(x)
  - 特殊的，若a=0，题目变成：
  - 对于二次函数f(x)，已知f(0)，f’(0)，f(a)，求f(x)

$$f(x)=\frac{f(a)-f'(0)a-f(0)}{a^2}x^2+f'(0)x+f(0)$$

从线性回归谈起

- $y=ax+b=ax_1+bx_0$，其中，$x_0\equiv 1$

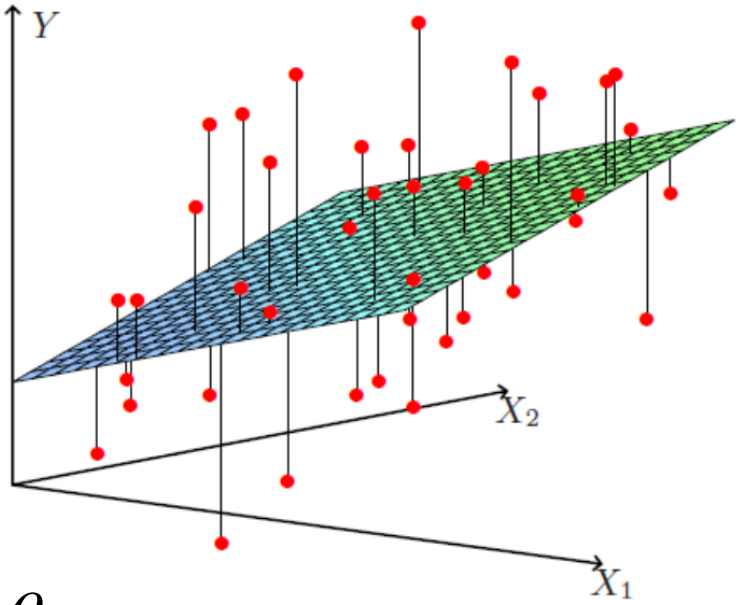




多个变量的情形

- 考虑两个变量

Living area (feet^2)	#bedrooms	Price (1000$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮



$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h_\theta(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x$$

θ的解析式的求解过程

- 将M个N维样本组成矩阵X：
  - 其中，X的每一行对应一个样本，共M个样本
  - X的每一列对应样本的一个维度，共N维
    - 其实还有额外的一维常数项，全为1
- 一个比较“符合常理”的误差函数为：

$$J(\theta)=\frac{1}{2}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)^2=\frac{1}{2}(X\theta-y)^T(X\theta-y)$$

- 已经论证：最小二乘建立的目标函数，是在高斯噪声的假设下，利用极大似然估计的方法建立的。

梯度下降算法 $J(\theta)=\frac{1}{2}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)^2$

- 初始化θ(随机初始化)
- 迭代，新的θ能够使得J(θ)更小
- 如果J(θ)无法继续减少或者达到循环上界次数，退出。

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

  - α：学习率、步长

线性回归目标函数的梯度方向计算

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_\theta(x) - y\right)^2 \\
&= 2 \cdot \frac{1}{2} \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x) - y\right) \\
&= \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\
&= \left(h_\theta(x) - y\right) x_j
\end{aligned}
$$

问题似乎完美解决

- 算法描述+凸函数极值

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

}

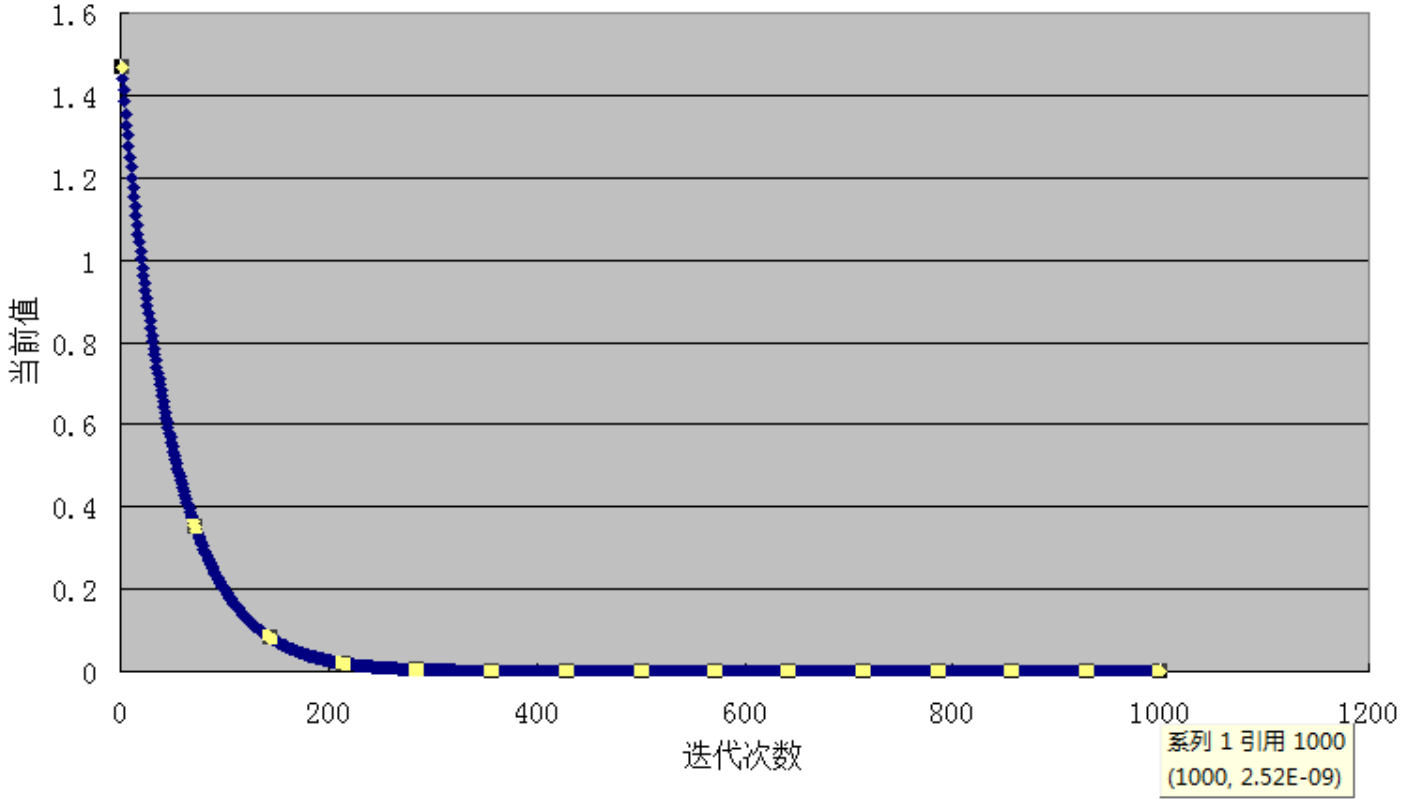
gradient descent. Note that, while gradient descent can be susceptible to local minima in general, the optimization problem we have posed here for linear regression has only one global, and no other local, optima; thus gradient descent always converges (assuming the learning rate α is not too large) to the global minimum. Indeed, J is a convex quadratic function.

思考

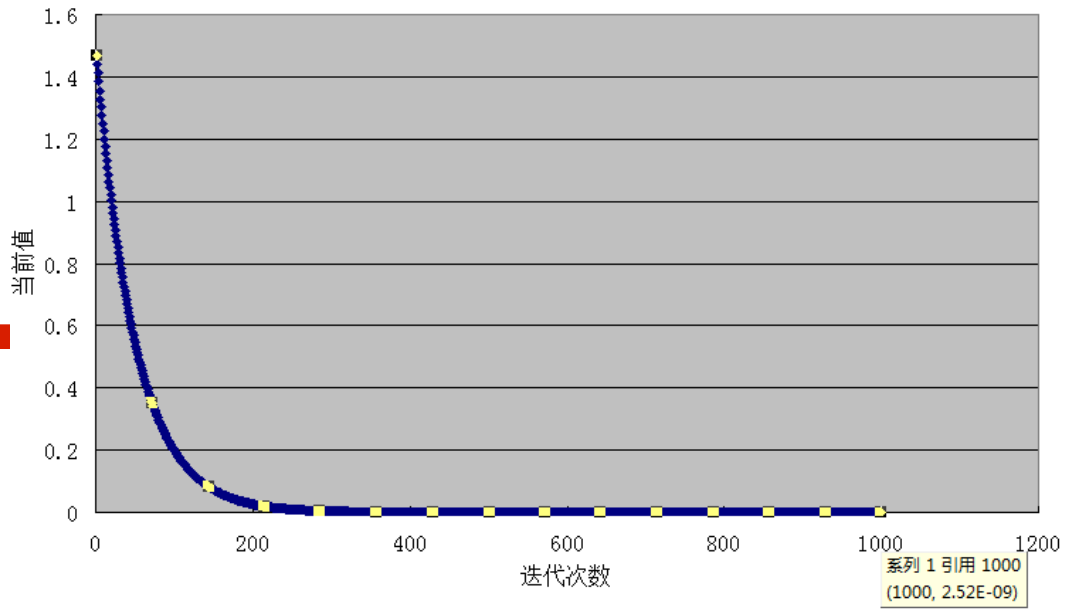
- 学习率α如何确定
  - 使用固定学习率还是变化学习率
  - 学习率设置多大比较好？
- 下降方向
  - 处理梯度方向，其他方向是否可以？
  - 可行方向和梯度方向有何关系？

实验：固定学习率的梯度下降

- $y=x^2$，初值取x=1.5，学习率使用0.01



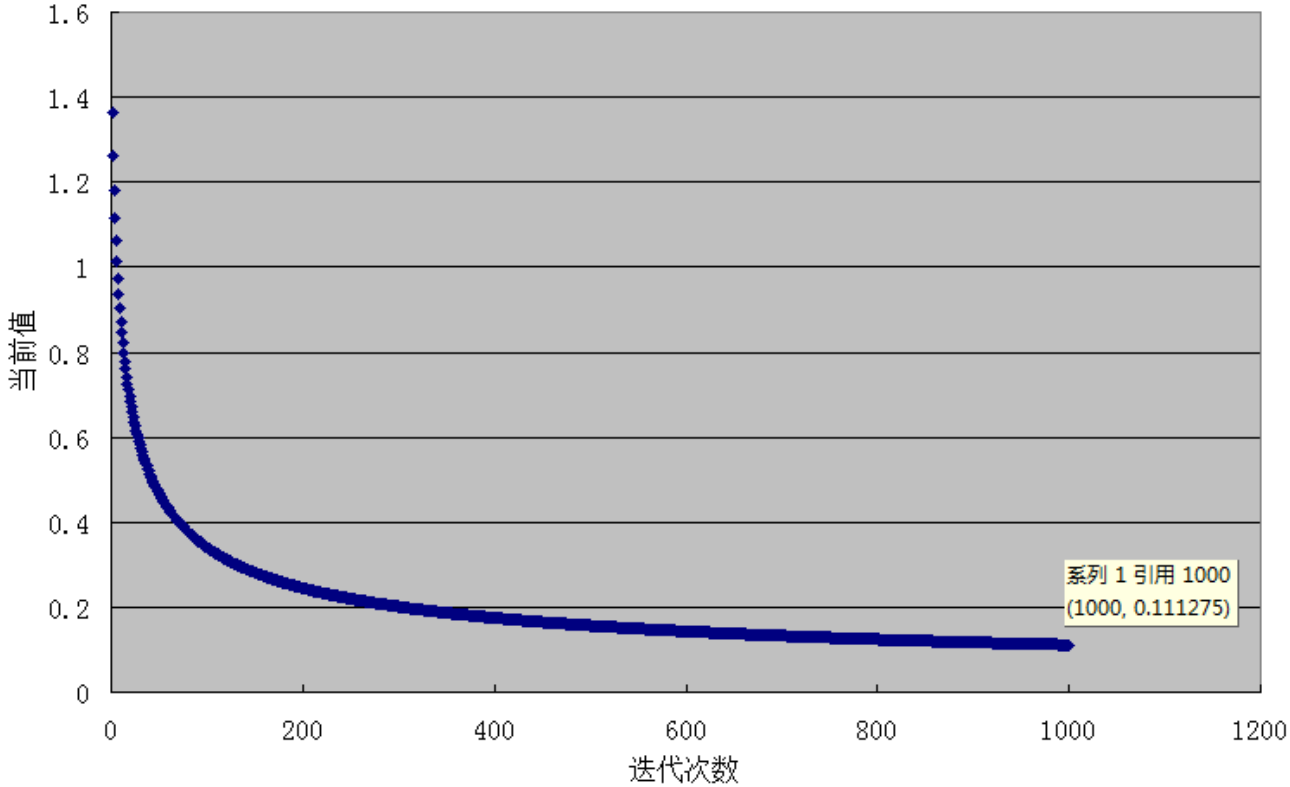
固定学习率



- 分析：
- 效果还不错
- 经过200次迭代，x=0.0258543；
- 经过1000次迭代，$x=2.52445\times10^{-9}$

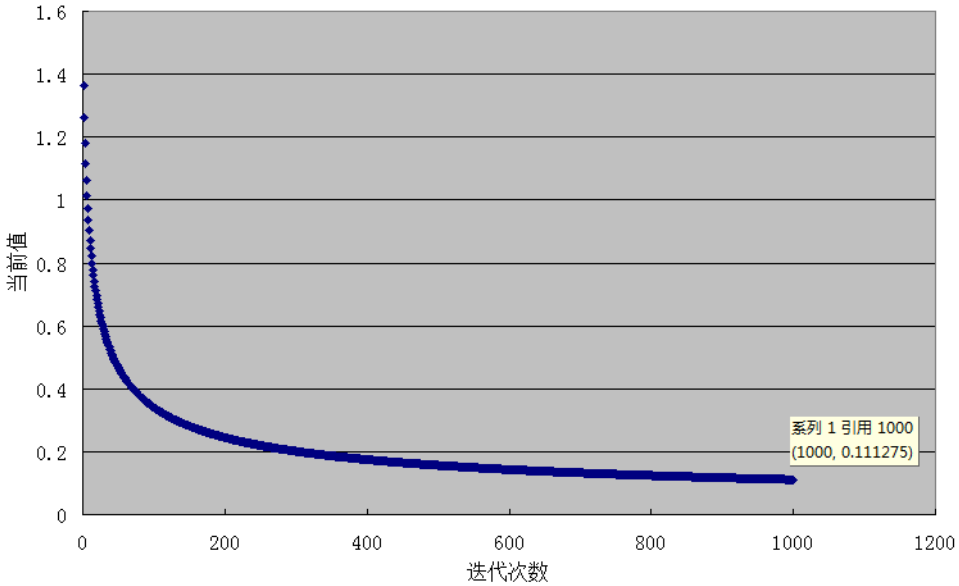
实验2：固定学习率的梯度下降

- $y=x^4$，初值取x=1.5，学习率使用0.01



固定学习率

- 分析：效果不理想
  - 经过200次迭代，x=0.24436；
  - 经过1000次迭代，x=0.111275



附：固定学习率实验的C代码

```
int _tmain(int argc, _TCHAR* argv[])
{
    double x = 1.5;
    double d;              //一阶导
    double a = 0.01;       //学习率
    for(int i = 0; i < 1000; i++)
    {
        d = g(x);
        x -= d * a;
        cout << i << '\t' << a << '\t' << x << '\n';
    }
    return 0;
}
```

优化学习率

- 分析“学习率α”在f(x)中的意义
- 调整学习率：
  - 在斜率(方向导数)大的地方，使用小学习率
  - 在斜率(方向导数)小的地方，使用大学习率
- 如何构造学习率α

梯度下降的运行过程分析

- x_k=a，沿着负梯度方向，移动到x_{k+1}=b，有：

$$b = a - \alpha \nabla F(a) \Rightarrow f(a) \geq f(b)$$

- 从x0为出发点，每次沿着当前函数梯度反方向移动一定距离αk，得到序列：

$$x_0, x_1, x_2, \cdots, x_n$$

- 对应的各点函数值序列之间的关系为：

$$f(x_0) \geq f(x_1) \geq f(x_2) \geq \cdots \geq f(x_n)$$

- 当n达到一定值时，函数f(x)收敛到局部最小值

视角转换

- 记当前点为$\mathbf{x_k}$，当前搜索方向为$\mathbf{d_k}$(如：负梯度方向)，因为学习率α是待考察的对象，因此，将下列函数$f(\mathbf{x_k}+\alpha\mathbf{d_k})$看做是关于α的函数h(α)。

$$h(\alpha)=f(x_k+\alpha d_k),\quad \alpha>0$$

- 当α=0时，$h(0)=f(\mathbf{x_k})$
- 导数 $\nabla h(\alpha)=\nabla f(x_k+\alpha d_k)^T d_k$

学习率α的计算标准

- 因为梯度下降是寻找f(x)的最小值，那么，在$\mathbf{x_k}$和$\mathbf{d_k}$给定的前提下，即寻找函数f($\mathbf{x_k}$+α$\mathbf{d_k}$)的最小值。即：

$$\alpha = \arg\min_{\alpha>0} h(\alpha) = \arg\min_{\alpha>0} f(x_k + \alpha d_k)$$

- 进一步，如果h(α)可导，局部最小值处的α满足：

$$h'(\alpha) = \nabla f(x_k + \alpha d_k)^T d_k = 0$$

学习率函数导数的分析 $h'(\alpha)=\nabla f(x_k+\alpha d_k)^T d_k=0$

- 将α=0带入：

$$h'(0)=\nabla f(x_k+0*d_k)^T d_k=\nabla f(x_k)^T d_k$$

- 下降方向$\mathbf{d_k}$可以选负梯度方向 $d_k=-\nabla f(x_k)$
  - 或者选择与负梯度夹角小于90°的某方向(后面会继续阐述搜索方向问题)
- 从而： $h'(0)<0$
- 如果能够找到足够大的α，使得 $h'(\hat{\alpha})>0$
- 则必存在某α，使得 $h'(\alpha^*)=0$
- α^*即为要寻找的学习率。

线性搜索(Line Search)

- 最简单的处理方式
  - 二分线性搜索(Bisection Line Search)
  - 不断将区间[α1, α2]分成两半，选择端点异号的一侧，知道区间足够小或者找到当前最优学习率。

回溯线性搜索(Backing Line Search)

- 基于Armijo准则计算搜素方向上的最大步长，其基本思想是沿着搜索方向移动一个较大的步长估计值，然后以迭代形式不断缩减步长，直到该步长使得函数值$f(\mathbf{x_k}+\alpha\mathbf{d_k})$相对与当前函数值$f(\mathbf{x_k})$的减小程度大于预设的期望值(即满足Armijo准则)为止。

$$f(x_k+\alpha d_k)\le f(x_k)+c_1\alpha\nabla f(x_k)^T d_k \qquad c_1\in(0,1)$$

回溯与二分线性搜索的异同

- 二分线性搜索的目标是求得满足h‘(α)≈0的最优步长近似值，而回溯线性搜索放松了对步长的约束，只要步长能使函数值有足够大的变化即可。
- 二分线性搜索可以减少下降次数，但在计算最优步长上花费了不少代价；回溯线性搜索找到一个差不多的步长即可。

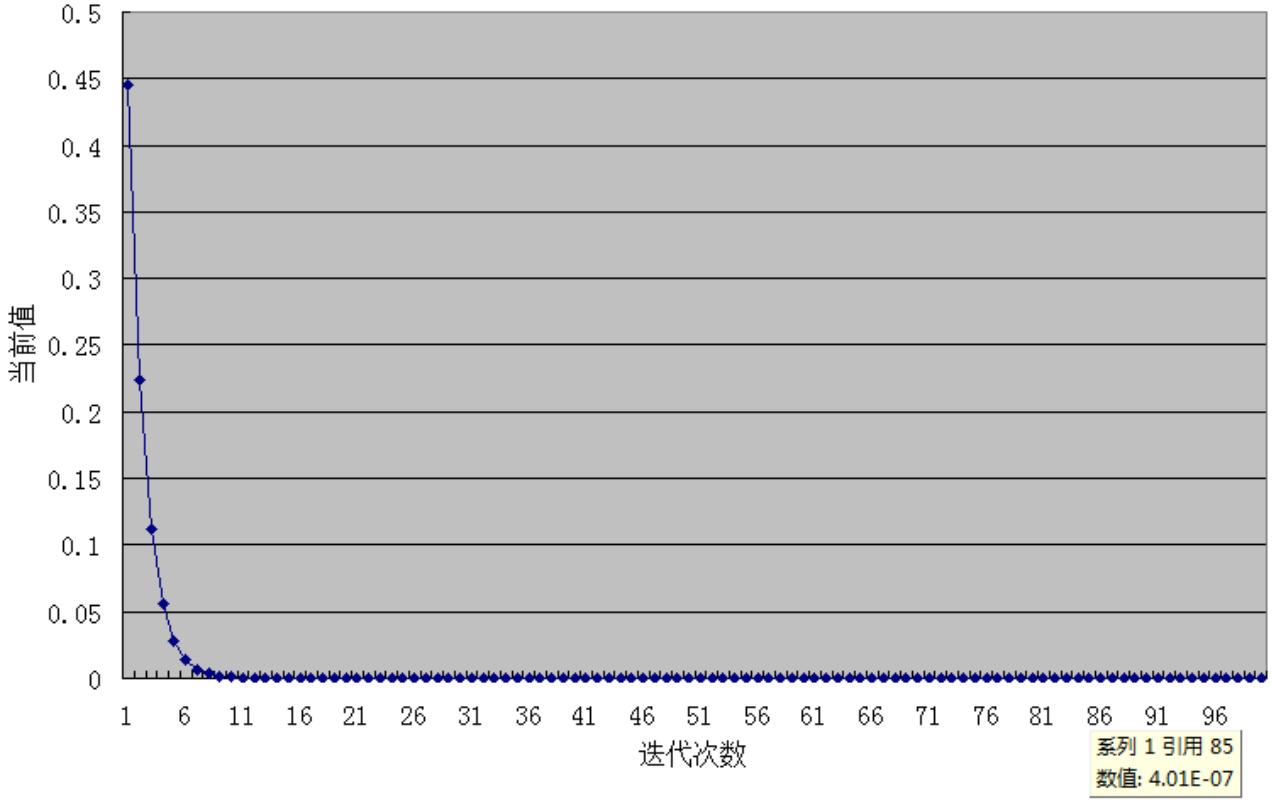
回溯线性搜索

- x为当前值
- d为x处的导数
- a为输入学习率
- 返回调整后的学习率

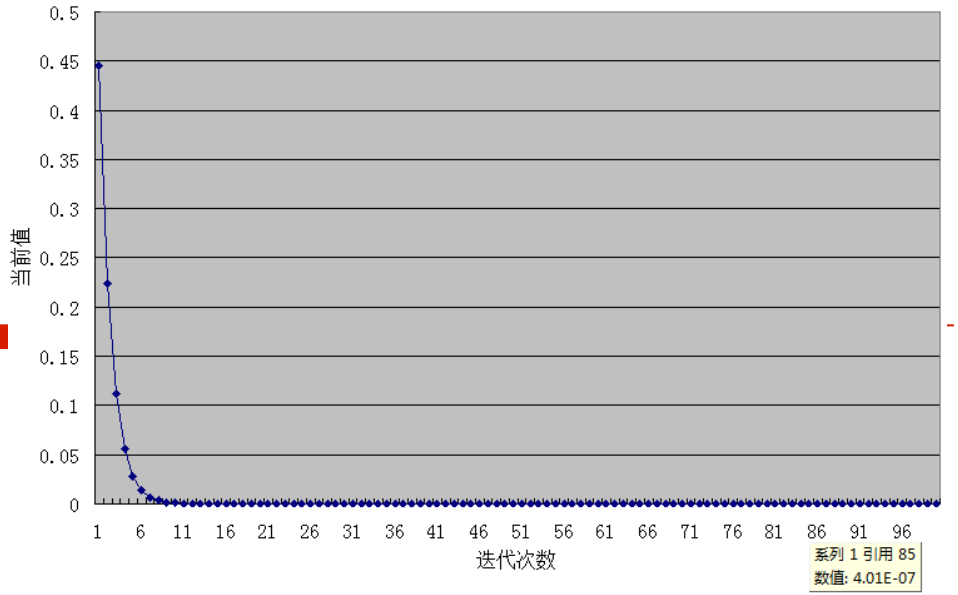
```
double GetA_Armijo(double x, double d, double a)
{
    double c1 = 0.3;
    double now = f(x);
    double next = f(x - a*d);

    int count = 30;
    while(next < now)
    {
        a *= 2;
        next = f(x - a*d);
        count--;
        if(count == 0)
            break;
    }

    count = 50;
    while(next > now - c1*a*d*d)
    {
        a /= 2;
        next = f(x - a*d);
        count--;
        if(count == 0)
            break;
    }
    return a;
}
```

实验：回溯线性搜索寻找学习率

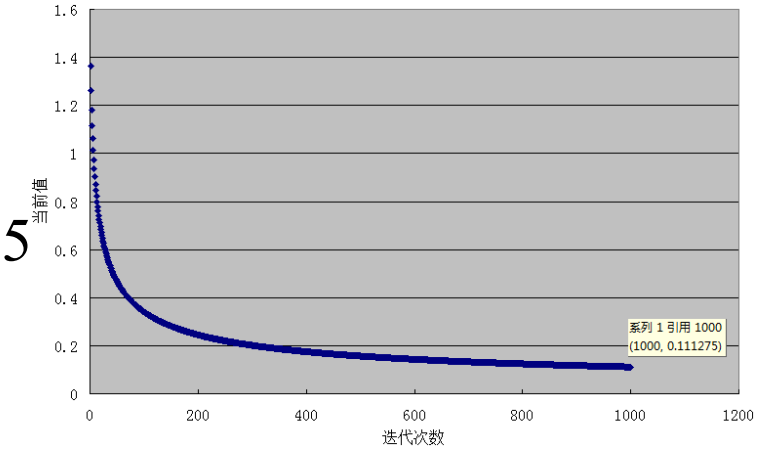
- $y=x^4$，初值取x=1.5，回溯线性方法



回溯线性搜索



- 分析：
- 效果还不错
- 经过12次迭代，x=0.00010872；
- 经过100次迭代，$x=3.64905\times10^{-7}$
  - 试比较固定学习率时:
    - 经过200次迭代，x=0.24436；
    - 经过1000次迭代，x=0.111275



回溯线性搜索的思考：插值法

- 采用多项式插值法(Interpolation) 拟合简单函数，然后根据该简单函数估计函数的极值点，这样选择合适步长的效率会高很多。
- 现在拥有的数据为：$\mathbf{x_k}$处的函数值$f(\mathbf{x_k})$及其导数$f'(\mathbf{x_k})$，再加上第一次尝试的步长α_0。如果α_0满足条件，显然算法退出；若α_0不满足条件，则根据上述信息可以构造一个二次近似函数：

$$h_q(\alpha)=\frac{h(\alpha_0)-h'(0)\alpha_0-h(0)}{\alpha_0^2}\alpha^2+h'(0)\alpha+h(0)$$

二次插值法求极值 $h_q(\alpha)=\frac{h(\alpha_0)-h'(0)\alpha_0-h(0)}{\alpha_0^2}\alpha^2+h'(0)\alpha+h(0)$

- 显然，导数为0的最优值为：

$$\alpha_1=\frac{h'(0)\alpha_0^2}{2\left[h'(0)\alpha_0+h(0)-h(\alpha_0)\right]}$$

- 若α_1满足Armijo准则，则输出该学习率；否则，继续迭代。

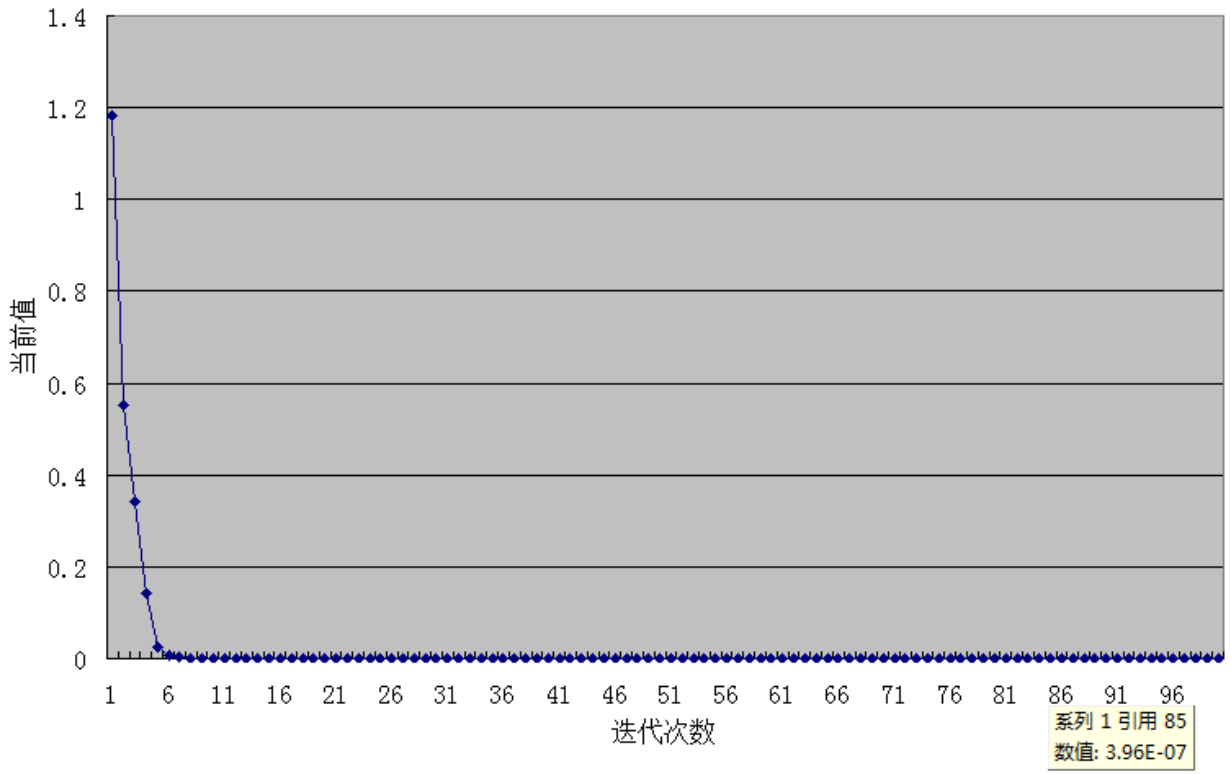
二次插值法

- x为当前值
- d为x处的导数
- a为输入学习率
- 返回调整后的学习率

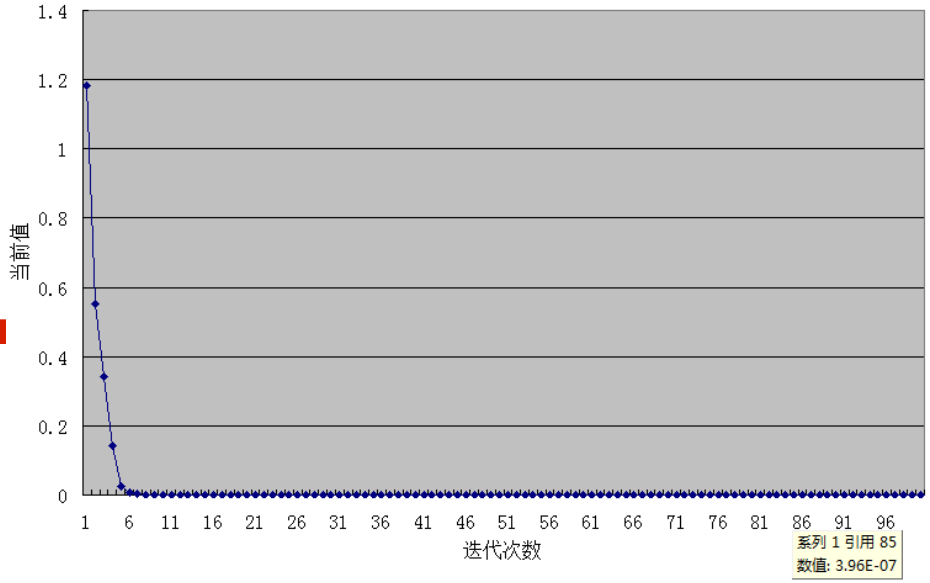
```
double GetA_Quad(double x, double d, double a)
{
    double c1 = 0.3;
    double now = f(x);
    double next = f(x - a*d);

    int count = 30;
    while(next < now)
    {
        a *= 2;
        next = f(x - a*d);
        count--;
        if(count == 0)
            break;
    }

    count = 50;
    double b;
    while(next > now - c1*a*d*d)
    {
        b = d * a * a / (now + d * a - next);
        b /= 2;
        if(b < 0)
            a /= 2;
        else
            a = b;
        next = f(x - a*d);
        count--;
        if(count == 0)
            break;
    }
    return a;
}
```

实验：二次插值线性搜索寻找学习率

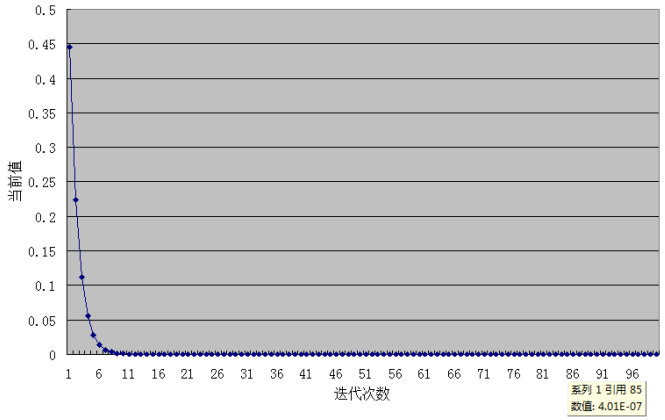
- $y=x^4$，初值取x=1.5，二次插值线性搜索方法



二次插值线性搜索



- 分析：
- 效果还不错
- 经过12次迭代，x=0.0000282229；
- 经过100次迭代，$x=3.61217\times10^{-7}$
  - 试比较回溯线性搜索时:
    - 经过12次迭代，x=0.00010872；
    - 经过1000次迭代，$x=3.649\times10^{-7}$



总结与思考

- 通过使用线性搜索的方式，能够比较好的解决学习率问题
- 一般的说，回溯线性搜索和二次插值线性搜索能够基本满足实践中的需要
- 问题：
  - 可否在搜索过程中，随着信息的增多，使用三次或者更高次的函数曲线，从而得到更快的学习率收敛速度？
  - 为避免高次产生的震荡，可否使用三次Hermite多项式，在端点保证函数值和一阶导都相等，从而构造更光顺的简单低次函数？

搜索方向

- 若搜索方向不是严格梯度方向，是否可以？
- 思考：
- 因为函数二阶导数反应了函数的凸凹性；二阶导越大，一阶导的变化越大。在搜索中，可否用二阶导做些“修正”？如：二者相除？

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

附：平方根算法

- 在任意点x_0处Taylor展开

令$f(x)=x^2-a$，即$f(x)=0$

$$\Rightarrow f(x)=f(x_0)+f'(x_0)(x-x_0)+o(x)$$

$$\Rightarrow f(x)\approx f(x_0)+f'(x_0)(x-x_0)$$

$$\Rightarrow 0=f(x_0)+f'(x_0)(x-x_0)$$

$$\Rightarrow x-x_0=-\frac{f(x_0)}{f'(x_0)}$$

$$\Rightarrow x=x_0-\frac{f(x_0)}{f'(x_0)}$$

将$f(x_0)=x_0^2-a$和$f'(x_0)=2x_0$带入，

$$\Rightarrow x=x_0-\frac{f(x_0)}{f'(x_0)}$$

$$\Rightarrow x=x_0-\frac{x_0^2-a}{2x_0}$$

$$\Rightarrow x=\frac{1}{2}\left(x_0+\frac{a}{x_0}\right)$$

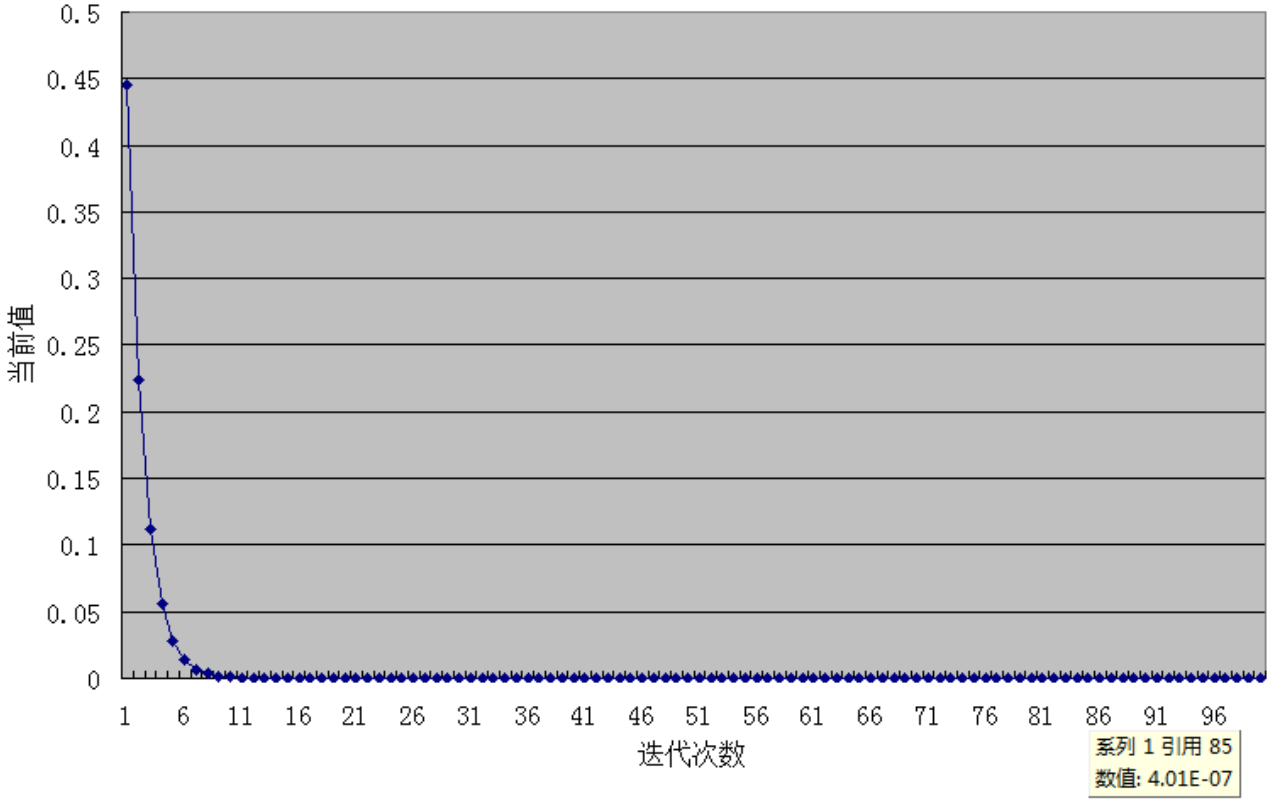
附：平方根

- 一般若干次(5、6次)迭代即可获得比较好的近似值。

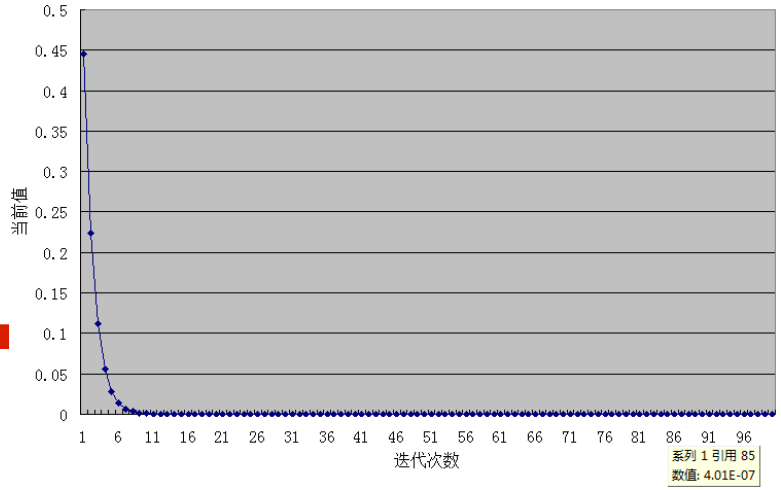
```
float Calc(float x);

int _tmain(int argc, _TCHAR* argv[])
{
    for(int i = 0; i <= 10; i++)
        cout << Calc((float)i) << '\n';
    return 0;
}

float Calc(float a)
{
    if(a < 1e-6)     //负数或者0，则直接返回0
        return 0;

    float x = a / 2;
    float t = a;
    while(fabs(x - t) > 1e-6)
    {
        t = x;
        x = (x + a/x) / 2;
    }
    return x;
}
```

实验：搜索方向的探索

- $y=x^4$，初值取x=1.5，负梯度除以二阶导



搜索方向的探索



- 效果出奇的好！
- 经过12次迭代，x=0.00770735；
- 经过100次迭代，$x=3.68948\times10^{-18}$
  - 试比较二次插值线性搜索时：
    - 经过12次迭代，x=0.0000282229；
    - 经过1000次迭代，$x=3.61217\times10^{-7}$

分析上述结果的原因

- 若f(x)二阶导连续，将f(x)在x_k处Taylor展开：

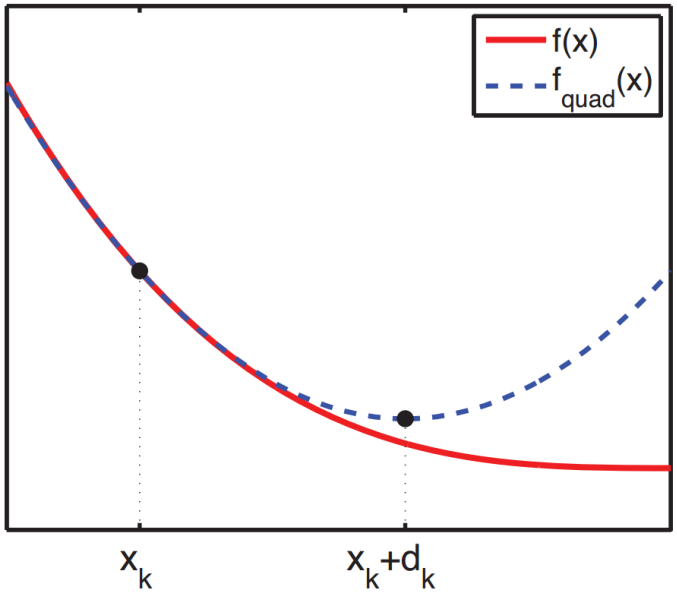
$$\varphi(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2} f''(x_k)(x - x_k)^2 + R_2(x)$$

$$\varphi'(x) \approx f'(x_k) + f''(x_k)(x - x_k)$$

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

牛顿法

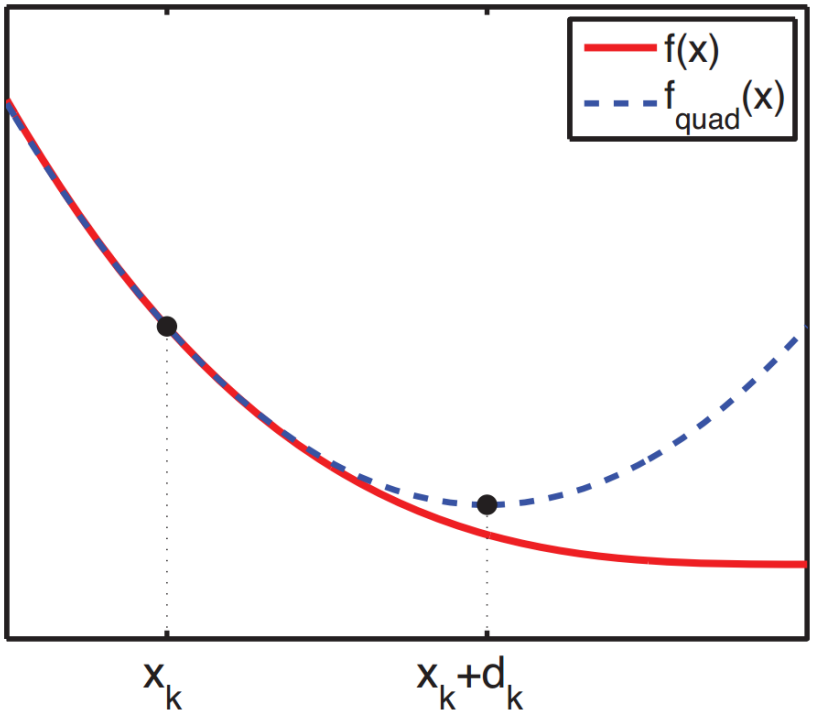
- 上述迭代公式，即牛顿法
- 该方法可以直接推广到多维：用方向导数代替一阶导，用Hessian矩阵代替二阶导

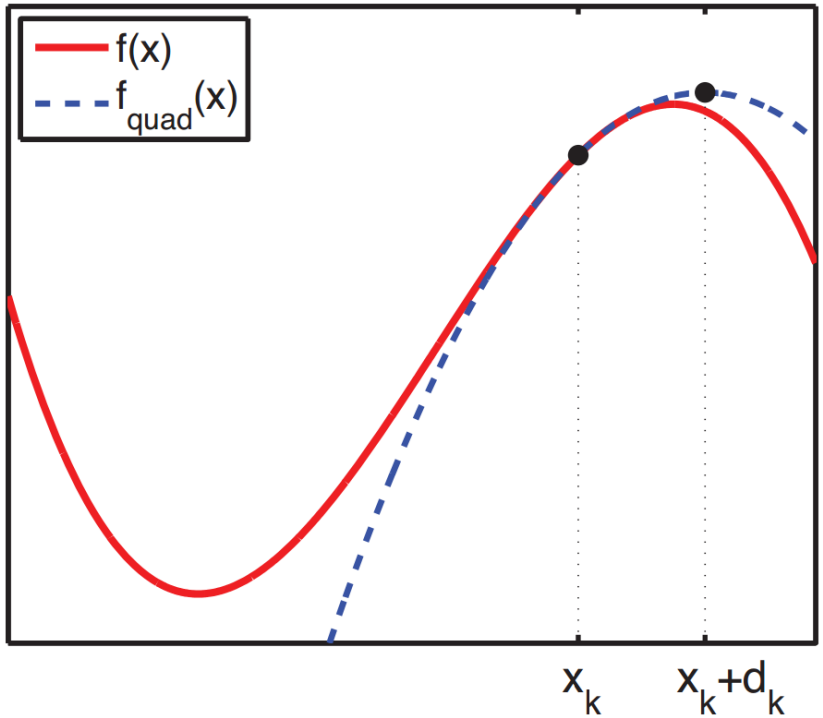


牛顿法的特点

- 经典牛顿法虽然具有二次收敛性，但是要求初始点需要尽量靠近极小点，否则有可能不收敛。
- 计算过程中需要计算目标函数的二阶偏导数，难度较大。
- 目标函数的Hessian矩阵无法保持正定，会导致算法产生的方向不能保证是f在x_k处的下降方向，从而令牛顿法失效；
- 如果Hessian矩阵奇异，牛顿方向可能根本是不存在的。

二阶导非正定的情况(一维则为负数)





修正牛顿方向

(1) Goldstein 和 Price 于 1967 年提出, 当 $\boldsymbol{G}_k$ 非正定时, 将搜索方向取为最速下降方向 $-\boldsymbol{g}^k$. 考虑到牛顿方向 $\boldsymbol{d}_k^N$ 与负梯度方向的夹角 $\langle \boldsymbol{d}_k^N, -\boldsymbol{g}^k \rangle$, 令搜索方向

$$\boldsymbol{d}^k = \begin{cases} \boldsymbol{d}_k^N, & \text{若 } \cos\langle \boldsymbol{d}_k^N, -\boldsymbol{g}^k \rangle \geqslant \eta > 0, \\ -\boldsymbol{g}^k, & \text{其他情形}, \end{cases}$$

其中 η 为某个事先设定的正数. 这样确定的搜索方向 $\boldsymbol{d}^k$ 满足 $\cos\langle \boldsymbol{d}^k, -\boldsymbol{g}^k \rangle \geqslant \eta > 0$, 从而可以保证算法的收敛性.

(2) Goldfeld 等人 (1966) 提出, 用正定矩阵 $\boldsymbol{G}_k + v_k \boldsymbol{I}$ 替代 Hesse 矩阵 $\boldsymbol{G}_k$, 计算修正牛顿方向. 比较理想的参数 $v_k > 0$ 满足: 适当大于使 $\boldsymbol{G}_k + v\boldsymbol{I}$ 正定的 "最小" 的 v. 此时, 可以借助于修正 Cholesky 分解算法确定参数 v_k.

拟牛顿的思路

- 求Hessian矩阵的逆影响算法效率，同时，搜索方向只要和负梯度的夹角小于90°即可，因此，可以用近似矩阵代替Hessian矩阵，只要满足该矩阵正定、容易求逆，或者可以通过若干步递推公式计算得到。
  - BFGS / LBFGS
  - Broyden – Fletcher – Goldfarb - Shanno

BFGS

- 矩阵迭代公式

$$
\begin{aligned}
\mathbf{B}_{k+1} &= \mathbf{B}_k + \frac{\mathbf{y}_k\mathbf{y}_k^T}{\mathbf{y}_k^T\mathbf{s}_k} - \frac{(\mathbf{B}_k\mathbf{s}_k)(\mathbf{B}_k\mathbf{s}_k)^T}{\mathbf{s}_k^T\mathbf{B}_k\mathbf{s}_k} \\
\mathbf{s}_k &= \boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1} \\
\mathbf{y}_k &= \mathbf{g}_k - \mathbf{g}_{k-1}
\end{aligned}
$$

  - 初值选单位阵 $\mathbf{B}_0 = \mathbf{I}$

- 记：$\mathbf{C}_k \approx \mathbf{H}_k^{-1}$
- 有：

$$
\mathbf{C}_{k+1} = \left(\mathbf{I} - \frac{\mathbf{s}_k\mathbf{y}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}\right)\mathbf{C}_k\left(\mathbf{I} - \frac{\mathbf{y}_k\mathbf{s}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}\right) + \frac{\mathbf{s}_k\mathbf{s}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}
$$

L-BFGS

- BFGS需要存储n×n的方阵C_k用于近似Hessian阵的逆矩阵；而L-BFGS仅需要存储最近m(m约为10，m=20足够)个 $(\Delta x, \Delta(\nabla f(x)))$ 用于近似C_k即可。
- L-BFGS的空间复杂度O(mn)，若将m看做常数，则为线性，特别适用于变量非常多的优化问题。

我们在这里

- 问题答疑：http://www.xxwenda.com/
  - 烦请邀请visio或其他人回答问题

- 本课程QQ群：
  - 471632108(小象学院：机器学习群)

感谢大家！

恳请大家批评指正！

附答案：花开堪折直须折

- 婚介所登记了N位男孩和N位女孩，每个男孩都对N个女孩的喜欢程度做了排序，每个女孩都对N个男孩的喜欢程度做了排序。你作为月老，能否给出稳定的牵手方案？
  - 分析：每个男孩都对N个女孩做了排序，N个男孩的钟情度形成N×N的矩阵A；同时，N个女孩的钟情度形成N×N的矩阵B；
  - 如果男孩i和女孩a牵手，但男孩i对女孩b更喜欢，而女孩b的男友j拼不过男孩i，则没有力量阻碍男孩i和女孩b的私奔，这即是不稳定的。

题目解析

- 采取贪心的思路：
- 所有男孩同时对各自最喜欢的女孩表白：
  - 若女孩a只有1个男孩i表白，接受男孩i；
  - 若女孩a有多位男孩表白，选择她最喜欢的男孩x
  - 若女孩a没有男孩表白，静静等待下一轮。
- 第k轮，所有单身男孩对各自第k喜欢的女孩表白：
  - 无论该女孩是否有男友；
  - 若女孩a收到了男孩i的表白：
    - 若女孩a没有男友，则接受男孩i；
    - 若相对于现任男友j，女孩a更喜欢男孩i，则接受男孩i；

正确性分析：匹配性和稳定性

- 在某个时刻，任何一个女孩要么已经有了男友，要么还会有男孩对其表白(她此刻没有男友，说明还有单身男孩，所以，该女孩不用着急，只需等待)，她总可以在表白的男孩中选择一个自己最喜欢的，因此，任何一个女孩必然都能找到男友。
  - 因为“N-N”的比例关系，这也导致了每个男孩都能有女友。
- 不稳定情况：如果男孩i和女孩a牵手，但男孩i对女孩b更喜欢，而女孩b的男友j拼不过男孩i，则男孩i和女孩b的私奔。
  - 1、男孩i和女孩a牵手
  - 2、男孩i对女孩b更喜欢
  - 3、相对于男友j，女孩b的更喜欢男孩i
  - 打破上述三个前提中的一个：1和2说明男孩i曾经向女孩b表白却遭拒，这说明相对于男孩i，女孩b的更喜欢现任男友j。

Code

```
int _tmain(int argc, _TCHAR* argv[])
{
    int man[N][N] = {
        {2,3,1,0},
        {2,1,3,0},
        {0,2,3,1},
        {1,3,2,0},
    };  //男孩喜欢的女孩列表
    int woman[N][N] = {
        {0,3,2,1},
        {0,1,2,3},
        {0,2,3,1},
        {1,0,3,2},
    };  //女孩喜欢的男孩列表
    int match[N];   //男孩的女友
    GaleShapley(man, woman, match);
    Print(match, N);
    Validate(man, woman, match);
    return 0;
}
```

```
const int N = 4;
void GaleShapley(const int (&man)[N][N], const int (&woman)[N][N], int (&match)[N])
{
    int wm[N][N];       //wm[i][j]: 女孩i对男孩j的排名
    int choose[N];      //choose[i]: 女孩i当前的男朋友
    int manIndex[N];    //manIndex[i]: 男孩i被多少个女孩拒绝过
    int i,j;
    int w,m;
    for(i = 0; i < N; i++)
    {
        match[i] = -1;      //所有男孩都初始化为光棍
        choose[i] = -1;     //所有女孩都初始化为光棍
        manIndex[i] = 0;    //为0，则意味着从最喜欢的女孩开始选
        for(j = 0; j < N; j++)
            wm[i][woman[i][j]] = j; //对男孩woman[i][j]的排名是第j位
    }

    bool bSingle = false;   //是否所有男孩都有了女友
    while(!bSingle)         //每个男孩当前轮选女友
    {
        bSingle = true;     //尚未发现单身男孩
        for(i = 0; i < N; i++)  //每个男孩i选择尚未被拒绝的最喜欢的女孩
        {
            if(match[i] != -1)  //男孩i已经有女友
                continue;
            bSingle = false;
            j = manIndex[i]++;
            w = man[i][j];      //男孩i第j喜欢的女孩w
            m = choose[w];      //女孩w当前的男友m
            if((m == -1) || (wm[w][i] < wm[w][m]))  //女孩w更喜欢男孩i
            {
                match[i] = w;
                choose[w] = i;
                if(m != -1)
                    match[m] = -1;  //女孩w抛弃前男友m
            }
        }
    }
}
```

改进相互表白的策略？

- 首轮a：所有男孩同时对各自最喜欢的女孩表白：
  - 若女孩a只有1个男孩i表白，接受男孩i；
  - 若女孩a有多位男孩表白，选择她最喜欢的男孩x
  - 若女孩a没有男孩表白，静静等待下一轮。
- 首轮b：所有女孩同时对各自最喜欢的男孩表白：
  - 策略同a
- 第k轮a：所有单身男孩对各自第k喜欢的女孩表白：
  - 无论该女孩是否有男友；
  - 若女孩a收到了男孩i的表白：
    - 若女孩a没有男友，则接受男孩i；
    - 若相对于现任男友j，女孩a更喜欢男孩i，则接受男孩i；
- 第k轮b：所有单身女孩对各自第k喜欢的男孩表白：
  - 策略同a

一个反例

- 男女孩各3位：
- 每个男孩的钟情度矩阵为：

	0	1	2
0	0	1	1
1	1	0	2
2	1	2	0

- 每个女孩的钟情度矩阵为：

	0	1	2
0	1	2	0
1	2	1	0
2	1	2	0

- 一个可行解为：2、0、1

算法的思考和总结

- 本算法能够解决这种情况吗？
  - 如果男孩i和女孩j牵手，但男孩i对女孩a更喜欢，而女孩a的男友b却更喜欢女孩j。
- 这即著名的对稳定婚姻策略的Gale-Shapley算法，由于女孩选择对其表白的男孩的最优者，因此也称为延迟认可算法。
  - 可用于学生志愿填报(员工选择职位、毕业生选择单位)等“二部”问题。
  - 如果没有“有向边”，则该算法不适用。